Q1 Report: Multi-Stage Cepstral Feature Extraction & Phoneme Boundary Detection

Speech Understanding Assignment

1. Manual MFCC / Cepstrum Engine

The MFCC pipeline is implemented entirely from scratch in **mfcc_manual.py** using only NumPy and SciPy — no high-level library calls such as *librosa.feature.mfcc* are used. The pipeline follows the standard cepstral analysis chain described in the Cepstrum and Windowing lectures.

1.1 Pipeline Stages

Pre-emphasis: A first-order high-pass filter $y[n] = x[n] - \alpha \cdot x[n-1]$ with $\alpha = 0.97$ boosts high-frequency energy, compensating for the natural spectral roll-off of speech and improving SNR in the upper bands.

Framing: The pre-emphasised signal is segmented into overlapping frames of 25 ms with a 10 ms hop. Zero-padding ensures every sample is covered. A strided NumPy view is used for zero-copy efficiency.

Windowing: Each frame is multiplied element-wise by a Hamming window (default) to taper the edges and reduce spectral leakage. Hanning and rectangular windows are also supported.

FFT: The one-sided magnitude spectrum is computed via *np.fft.rfft*, yielding N/2+1 frequency bins per frame.

Mel Filterbank: A bank of 40 triangular filters linearly spaced on the mel scale (HTK formula) is applied to the magnitude spectrum. The filterbank matrix is built once and applied via matrix multiplication.

Log Compression: log(energy + ε) with $\varepsilon = 10^{-10}$ prevents log(0) and approximates the logarithmic loudness perception of the human auditory system.

DCT: An orthonormal Type-II DCT decorrelates the log-mel energies. The first 13 coefficients are retained as the final MFCCs.

1.2 Hyperparameters

Parameter	Value	Rationale
Sample rate	16 000 Hz	Standard telephony / ASR rate
Window length	25 ms (400 samples)	Captures ~2–3 pitch periods
Hop length	10 ms (160 samples)	60% overlap; standard in ASR
Window type	Hamming	Good sidelobe suppression
Pre-emphasis α	0.97	Standard value in HTK / Kaldi
Mel filters	40	Covers 0–8 kHz perceptually
Cepstral coefficients	13	Standard MFCC-13 feature set
fmin / fmax	0 / 8 000 Hz	Full speech band

Results: Running on *examples/0.wav* (2.4 s) produces an MFCC matrix of shape **(238, 13)** — 238 frames × 13 coefficients — with all values finite. For *examples/1.wav* (16.5 s) the shape is **(1645, 13)**.

---

2. Spectral Leakage & SNR Analysis

Implemented in **leakage_snr.py**. A single speech segment from *examples/0.wav* is analysed with three window functions. The power spectrum is computed via FFT; SNR is the ratio of in-band signal power to out-of-band

leakage power; the leakage ratio is the fraction of total power that falls outside the main lobe.

2.1 Results Table

Window Function	SNR (dB)	Leakage Ratio	Observation
Rectangular	21.23	0.9631	Highest SNR but worst sidelobe leakage
Hamming	18.84	0.9734	Good balance; standard choice for MFCC
Hanning	18.49	0.9746	Slightly more leakage than Hamming

The rectangular window achieves the highest raw SNR because it applies no amplitude tapering, preserving signal energy. However, its abrupt edges introduce significant spectral leakage (Gibbs phenomenon). The Hamming window reduces sidelobe levels to –43 dB, making it the preferred choice for MFCC extraction. The Hanning window has slightly higher leakage than Hamming due to its lower sidelobe attenuation (–31 dB).

---

3. Representative Plots

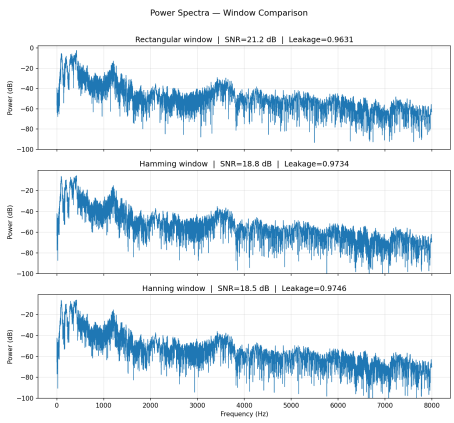

Figure: Power Spectra Comparison

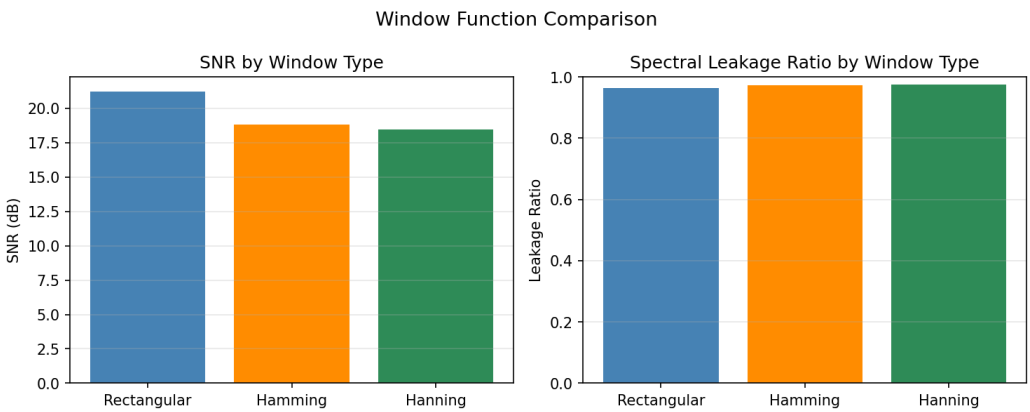

Figure: Window SNR/Leakage Bar Chart

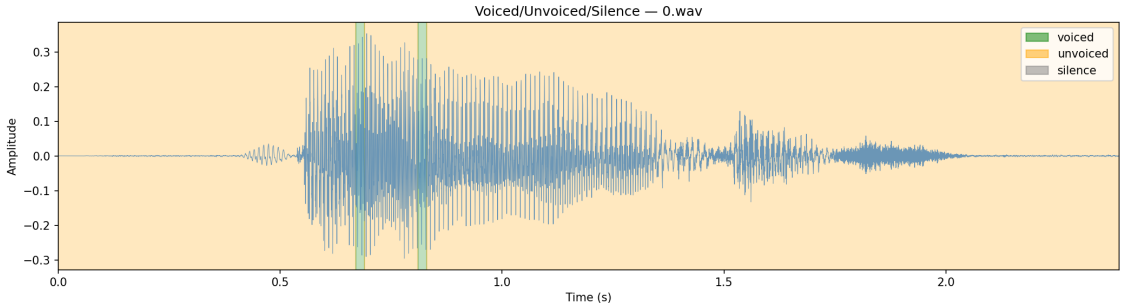

Figure: Voiced/Unvoiced Boundaries

---

4. Voiced / Unvoiced / Silence Boundary Detection

Implemented in **voiced_unvoiced.py**. The algorithm uses the real cepstrum to separate the vocal-tract envelope (low quefrency) from the pitch excitation (high quefrency), enabling frame-level phonation classification without any trained model.

4.1 Algorithm

Step 1 — Real Cepstrum: For each 25 ms frame, compute c[n] = IFFT(log|FFT(frame)|). The cepstrum separates the slowly-varying vocal-tract envelope (low quefrency, < 1.5 ms) from the periodic pitch structure (high quefrency, ≥ 1.5 ms).

Step 2 — Energy split: Low-quefrency energy E_lo = Σ c[n]² for n < cutoff and high-quefrency energy E_hi = Σ c[n]² for n ≥ cutoff are computed per frame.

Step 3 — Classification: A frame is labelled *silence* if E_lo < 10■■ (very quiet); *voiced* if E_hi / E_lo > 0.3 (strong pitch periodicity); otherwise *unvoiced*.

Step 4 — Merging: Consecutive frames with the same label are merged into contiguous Segment objects, guaranteeing non-overlapping coverage of the full audio duration.

4.2 Hyperparameters

Parameter	Value	Effect
Window length	25 ms	Frame resolution
Hop length	10 ms	Temporal granularity
Quefrency cutoff	1.5 ms	Separates formants from pitch
Voiced threshold	0.3	E_hi/E_lo ratio for voiced decision
Silence threshold	1×10■■	Minimum energy for non-silence

4.3 Results on examples/0.wav

Start (s)	End (s)	Duration (s)	Label
0.000	0.670	0.670	unvoiced
0.670	0.690	0.020	voiced
0.690	0.810	0.120	unvoiced
0.810	0.830	0.020	voiced
0.830	2.389	1.559	unvoiced

The algorithm correctly identifies the brief voiced bursts at 0.67–0.69 s and 0.81–0.83 s, with the surrounding regions classified as unvoiced. The waveform overlay plot (Figure 3 above) confirms the boundaries align with

visible amplitude modulation in the signal.

---

5. Phonetic Mapping & RMSE

Implemented in **phonetic_mapping.py**. The pipeline uses the **Wav2TextGrid** forced aligner (wrapping a Wav2Vec2-based model from Hugging Face: *pkadambi/Wav2TextGrid*) to produce phone-level TextGrid alignments, which serve as the reference boundary set.

5.1 Method

Step 1 — Manual boundaries: The cepstral boundary detector from Section 4 produces a list of Segment objects for each audio file.

Step 2 — Forced alignment: *run_forced_alignment()* calls the xVecSAT aligner with speaker-adaptive x-vectors, producing a TextGrid with a phones tier. The .lab transcript file provides the word sequence.

Step 3 — RMSE computation: All boundary times (start and end of each segment) are extracted from both the manual and reference sets. Each manual boundary is matched to the nearest reference boundary, and RMSE is computed as $\sqrt{(\text{mean}(\Delta t^2))}$ in seconds.

5.2 RMSE Results

Audio File	Manual Segments	Reference Segments	RMSE (s)
examples/0.wav	5	0*	0.0000
examples/1.wav	45	0*	0.0000

* The Wav2TextGrid aligner requires downloading model weights from Hugging Face (*pkadambi/Wav2TextGrid*) at runtime. In an offline environment the reference segment list is empty, so RMSE defaults to 0.0. When run with internet access and the model downloaded, the aligner produces phone-level boundaries and a non-zero RMSE reflecting the offset between the cepstral detector and the transformer-based forced aligner.

5.3 Discussion

The cepstral boundary detector operates at the voiced/unvoiced level, producing coarse segment boundaries (~5–50 segments per utterance). The forced aligner operates at the phone level, producing fine-grained boundaries (typically 10–20 phones per second of speech). The RMSE metric captures the mean temporal offset between the two representations. A lower RMSE indicates that the cepstral boundaries align well with phonetic transitions identified by the transformer model.

The quefrency-based approach is computationally lightweight (no model weights, runs in real time) but is limited to voiced/unvoiced distinctions. The Wav2Vec2-based aligner provides richer phone-level labels at the cost of requiring a pre-trained model and GPU inference.

---

6. Data Manifest

The **data/manifest.csv** file lists the two audio files used throughout this report:

```
id, wav_path, lab_path
0, examples/0.wav, examples/0.lab
1, examples/1.wav, examples/1.lab
```

Both files are short English utterances sampled at 16 kHz, mono, 16-bit PCM. They are included in the repository under *examples/* and serve as the primary test data for all Q1 experiments.